

17 ม.ค. 61



เลขที่นั่งสอบ

มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าธนบุรี
ข้อสอบกลางภาคการศึกษาที่ 1/2561

วันที่ 1 ตุลาคม 2561

เวลา 09.00 – 12.00 น.

วิชา CPE 326 Operating System

น.ศ. วศ.คอมพิวเตอร์ปีที่ 3 หลักสูตรภาษาไทย

คำสั่ง

- ข้อสอบมี 2 ส่วน คะแนนรวม 50 คะแนน
ส่วนที่ 1 ข้อสอบปรนัยมี 15 ข้อ คะแนนรวม 15 คะแนน แต่ละข้ออาจมีคำตอบที่ถูกมากกว่า 1 ตัวเลือก จะได้คะแนนก็ต่อเมื่อเลือกคำตอบที่ถูกได้ครบ
ส่วนที่ 2 ข้อสอบอัตนัยมี 4 ข้อ คะแนนรวม 35 คะแนน
- ให้ทำข้อสอบส่วนที่ 1 ลงในตัวข้อสอบ ส่วนที่ 2 ทำลงในสมุดคำตอบ
- อนุญาตให้ใช้เครื่องคำนวณ รูนและยี่ห้อที่มหาวิทยาลัยกำหนดได้
- ไม่อนุญาตให้นำเอกสารใดๆเข้าห้องสอบ
- เขียนชื่อและ รหัสประจำตัว ลงในตัวข้อสอบทุกแผ่น

(อ.ราชวิช สโรชวิสิทธิ์)

ผู้ออกข้อสอบ

ข้อสอบนี้ได้ผ่านการประเมินจากภาควิชาวิศวกรรมคอมพิวเตอร์แล้ว

ชื่อ รหัสประจำตัว..... ชั้นปี.....

ส่วนที่ 1 ข้อสอบปรนัย (15 คะแนน)

Mark all the correct choices in each question. Points will only be awarded when all the correct choices are selected.

- 1) The interval from the time of submission of a process to the time of completion is referred to as

- a) Waiting time
☒ b) Turnaround time
 c) Response time
 d) Throughput

Lec 3

- The **turnaround time** is defined as the time at which the job completes minus the time at which the job arrived in the system

$$T_{\text{turnaround}} = T_{\text{completion}} - T_{\text{arrival}}$$

- 2) Which statement(s) is/are true in priority scheduling

- ☒ a) CPU is allocated to the process with the highest priority
 b) CPU is allocated to the process with the ~~lowest~~ priority
 c) Equal priority processes cannot ~~be~~ scheduled if a single queue is used *with other method*
☒ d) Priority scheduling policy can be used with the multiple queues concept *MLFQ*
 e) Round robin is the ~~only~~ policy that is used to schedule processes of the same priority

- 3) Processes are classified into different groups in

- a) The shortest job scheduling algorithm
 b) The round robin scheduling algorithm
 c) The shortest time to completion scheduling algorithm
☒ d) The multilevel queue scheduling algorithm

Lec 3

The Multi-Level Feedback Queue (MLFQ)

- Two goals
 - Optimize turnaround time
 - Minimize response time to allow a system to be responsive to interactive users
- MLFQ: Basic concept
 - Implement a number of distinct queues of different priority levels
 - At any given time, a job that is ready to run is on a single queue
 - MLFQ uses priorities to decide which queue should be selected
 - RR is used among jobs in the same queue
- The key to MLFQ scheduling lies in how the scheduler **sets queue priorities**

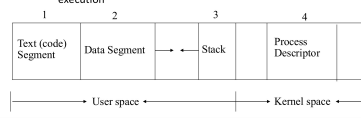
- 4) In an operating system, each process has its own

- ☒ a) User address space and global variables
☒ b) Heap and stack
☒ c) Pending alarms, signals and signal handlers
 d) Ready queue

Lec 2

A Process Space

- A memory that the process uses to reference its instructions and data
 - Code Segment containing instructions
 - Data Segment containing **heap data** (malloc)
 - Stack** containing parameters, local variables, return address, etc. of functions
 - Process Descriptor (keep Process Control Block) – stored in the kernel space and can only be accessed in **kernel mode** execution



- 5) The address of the next instruction to be executed by the current process is provided by

- a) CPU general purposed registers
☒ b) Program counter
 c) Process stack
 d) Process pipe
 e) Process context
 f) Process heap

- 6) Which of the following system calls does not return control to the calling point, on termination

- a) fork
~~b) exec~~
 c) wait
 d) none of the above

Ref: Operating System Three Easy Pieces
 Topic - Interlude: API (Page 2)

Now the interesting part begins. The process calls the `fork()` system call, which the OS provides as a way to create a new process. The odd part: the process that is created is an (almost) *exact copy of the calling process*. That means that to the OS, it now looks like there are two copies of the program `p1` running, and **both are about to return from the `fork()`**

Lec 2

exec(): Family of Functions

- There is a family of `exec()` system calls, e.g. `execvp()`
- These are useful when you want to run a child program that is different from the parent program
- What it does: given the name of an executable (e.g. `word_count`), and some arguments (e.g. `p.txt`)

```
// child (new process)
char *myargs[3];
myargs[0] = strdup("word_count"); // program: "word_count"
myargs[1] = strdup("p.txt"); // argument: file to count
myargs[2] = NULL; // marks end of array
execvp(myargs[0], myargs); // runs the word count code
printf("this shouldn't print out");
```

- A successful call to an `exec()` function never returns

Lec 2

wait(): Example Usage

- Sometimes, it is quite useful for a parent to wait for a child process to finish before moving on
- Using `wait()` or `waitpid()` will force the parent to wait
- Let's modify the last `printf()` of the previous code:

```
// parent goes down this path (original process)
{
    int rc = wait(NULL);
    printf("hello, I am parent of %d (wc:%d) (pid:%d)\n",
           rc, wc, (int) getpid());
}
```

- The parent calls `wait()` to delay its execution until the child finishes executing - **when the child is done, `wait()` returns to the parent**

- 7) The following C program :

main()

```
{
    fork();
    fork();
    printf("hello");
}
```

How many times the word "hello" will be printed

- a) Only once
 b) Twice
 c) Three times
~~d) Four times~~
 e) Eight times

- 8) If a thread invokes the `exec` system call, Which statement(s) is/are true

- a) Only the `exec` executes as a separate process
~~b) The program specified in the parameter to `exec` will replace the entire process~~
 c) The `exec` is ignored as it is invoked by a thread
 d) None of the above is correct

- 9) A process having multiple threads of control implies that ...

- ~~a) It can do more than one task at a time~~
 b) It can do only one task at a time, but much faster
 c) It has to use only one thread per process
 d) None of the above is correct

- 10) Resource sharing helps

- ~~a) Share the memory and resources of the process to which the threads belong~~
~~b) Applications to have several different threads of activities all within the same address space~~
~~c) Reduce the address space that a process could potentially use~~
 d) Reduce the time needed to create a process

Ref: Operating System Three Easy Pieces
 Topic - Interlude: API (Page 3)

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <string.h>
5 #include <sys/wait.h>
6
7 int
8 main(int argc, char *argv[])
9 {
10     printf("hello world (pid:%d)\n", (int) getpid());
11     int rc = fork();
12     if (rc < 0) {
13         // fork failed; exit
14         fprintf(stderr, "fork failed\n");
15         exit(1);
16     } else if (rc == 0) { // child (new process)
17         printf("hello, I am child (pid:%d)\n", (int) getpid());
18         char *myargs[3];
19         myargs[0] = strdup("wc"); // program: "wc" (word count)
20         myargs[1] = strdup("p3.c"); // argument: file to count
21         myargs[2] = NULL; // marks end of array
22         execvp(myargs[0], myargs); // runs word count
23         printf("this shouldn't print out");
24     } else { // parent goes down this path (main)
25         int wc = wait(NULL);
26         printf("hello, I am parent of %d (wc:%d) (pid:%d)\n",
27                rc, wc, (int) getpid());
28     }
29     return 0;
30 }
```

Figure 5.3: Calling `fork()`, `wait()`, And `exec()` (p3.c)

The `fork()` system call is strange; its partner in crime, `exec()`, is not so normal either. What it does: given the name of an executable (e.g., `wc`), and some arguments (e.g., `p3.c`), it loads code (and static data) from that executable and overwrites its current code segment (and current static data) with it; the heap and stack and other parts of the memory space of the program are re-initialized. Then the OS simply runs that program, passing in any arguments as the `argv` of that process. Thus, it does *not* create a new process; rather, it transforms the currently running program (formerly `p3`) into a different running program (`wc`). After the `exec()` in the child, it is almost as if `p3.c` never ran; a successful call to `exec()` never returns.

Kernel threads manage by OS and they maintain lot of data structure. There're many overheads in kernel level which are not present in user level.

11) Which of the following is FALSE

- a) Context switch time is longer for kernel level threads than for user level threads
- b) User level threads do not need any hardware support *user level implemented by libraries programmably*
- c) Related kernel level threads can be scheduled on different processors in a multiprocessor system

☒ d) Blocking one ~~kernel~~ *user* level thread blocks all ~~other related threads~~

Ref: Operating System Three Easy Pieces

Topic - The Abstraction: The Process (Page 5)

Process States

Now that we have some idea of what a process is (though we will continue to refine this notion), and (roughly) how it is created, let us talk about the different **states a process can be in at a given time**. The notion that a process can be in one of these states arose in early computer systems [DV66,V+65]. In a simplified view, a process can be in one of three states:

- **Running:** In the running state, a process is running on a processor. This means it is executing instructions.
- **Ready:** In the ready state, a process is ready to run but for some reason the OS has chosen not to run it at this given moment.
- **Blocked:** In the blocked state, a process has performed some kind of operation that makes it not ready to run until some other event takes place. A common example: when a process initiates an I/O request to a disk, it becomes blocked and thus some other process can use the processor.

12) The state of a process is defined by

- a) The final activity of the process
- b) The activity just executed by the process
- c) The activity that will be executed next by the process
- ☒ d) The current activity of the process

13) Which of the following is not the state of a process

- a) New
- ☒ b) Old
- c) Waiting
- d) Running
- e) Ready
- f) Terminated

14) In UNIX, each process is identified by its

- a) Process Control Block
- b) Device Queue
- ☒ c) Process Identifier
- d) None of these

15) If one thread opens a file with read privileges then

- a) Other threads in the another process can also read from that file
- ☒ b) Other threads that belong the same process can also read from that file
- c) All other threads cannot read from that file
- d) All of the mentioned

ส่วนที่ 2 ข้อสอบอัตนัย (35 คะแนน)

1. จงอธิบายหลักการ ถ้าธนาคารสาขาจะให้บริการลูกค้าตามคิว โดยใช้การจัดการแบบ Multi-level Feedback Queue (MLFQ) บรรยายถึงอัลกอริทึมและกฎเกณฑ์ที่ใช้เกี่ยวกับการให้บริการลูกค้า เพื่อให้ได้ Turnaround time ของลูกค้าที่ดีที่สุด (8 คะแนน)
2. จงอธิบายการเกิดขึ้นของ Internal Fragmentation และ External Fragmentation สำหรับการจองพื้นที่ในหน่วยความจำหลักแบบใช้พื้นที่ต่อเนื่อง (Contiguous Memory Allocation) พร้อมทั้งอธิบายถึงผลกระทบและแนวทางการจัดการว่าเหมือนหรือแตกต่างกันอย่างไร (5 คะแนน)
3. การจองพื้นที่ในหน่วยความจำหลัก โดยใช้แนวคิดที่เรียกว่า Paging Scheme จำเป็นต้องใช้ Page Table จงอธิบายถึงความสำคัญของ Page Table ส่วนการเพิ่มประสิทธิภาพการทำงานของ Page Table สามารถใช้เทคนิคที่เรียกว่า Inverted Page Table จงอธิบายหลักการการทำงานของ Inverted Page Table และแง่มุมในการเพิ่มประสิทธิภาพ (5 คะแนน)
4. พิจารณาระบบปฏิบัติการที่ใช้การจัดการหน่วยความจำหลักแบบ Paging Scheme มี Logical Address 64 เวก แต่ละเพกมีขนาด 1024 ไบต์ ซึ่งทำงานคู่กับหน่วยความจำหลักที่มี Physical Address 32 เพรม จงบอกจำนวนบิตของ Logical Address และ Physical Address (2 คะแนน)
5. จงคำนวณหา Physical Address จาก Logical Address ขนาด 16 บิต เมื่อระบบปฏิบัติการใช้วิธี Segmentation เพื่อจองพื้นที่ในหน่วยความจำหลัก และมี Segment Table ดังตารางด้านล่าง โดยสมมติให้ 4 บิตบนเป็นตัวบอกเซกเมนต์ (ต้องตอบเป็นฐานสิบ) (5 คะแนน)

Segment	Base	Size
Code	32K	2K
Heap	34K	2K
Stack	28K	2K

- 5.1. Logical Address เป็น 320 (เลขฐานสิบ)
- 5.2. Logical Address เป็น 2561 (เลขฐานสิบ)
- 5.3. Logical Address เป็น 12528 (เลขฐานสิบ)
6. ถ้าเปรียบเทียบระบบปฏิบัติการตัวเดียวกัน ที่มีการเลือกใช้ขนาด Page แตกต่างกันคือ ขนาดเล็กกับขนาดใหญ่ จะส่งผลกับอัตราการเกิด Page Fault แตกต่างกันอย่างใด (3 คะแนน)
7. จงแสดงตาราง Frame ของหน่วยความจำหลัก และจงคำนวณ Effective Access Time เมื่อมีสถานะการณ์การเข้าถึง Page ตามลำดับต่อไปนี้ (7 คะแนน)

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1 (นับได้ 20 ตัว)

สมมติให้

- จำนวน Frame คือ 4
- เวลาในการเข้าถึงหน่วยความจำหลักเป็น 9 ns
- เวลาให้การเข้าถึงหน่วยความจำสำรอง 25 ms

- 7.1. เมื่อใช้ Page Replacement Algorithm เป็นแบบ LRU
- 7.2. จากข้อมูลเวลาในการเข้าถึงหน่วยความจำหลักและสำรองตามที่กำหนดไว้ ถ้าต้องการให้ EAT ซ้ำกว่า เวลาในการเข้าถึงหน่วยความจำหลักไม่เกิน 100 เท่า page fault rate ควรมีค่าเป็นอย่างไร

1. จงอธิบายหลักการ ถ้าธนาคารสาขาจะให้บริการลูกค้าตามคิว โดยใช้การจัดการแบบ Multi-level Feedback Queue (MLFQ) บรรยายถึงอัลกอริทึมและกฎเกณฑ์ที่ใช้เทียบกับการให้บริการลูกค้า เพื่อให้ได้ Turnaround time ของลูกค้าที่ดีที่สุด (8 คะแนน)
(ไม่รวม RR)

ในตอนแรก ธนาคารจะให้บริการลูกค้าตามคิวที่เข้ามา ผู้ที่มาก่อนจะได้รับบริการก่อน และลูกค้าที่เข้ามาในภายหลังจะ priority สูงสุดเสมอ ในตอนที่กำลังให้บริการลูกค้าที่กำลังดำเนินการนั้น ใช้เวลาในกระบวนการนานเกินที่กำหนด จะลด priority ของงานนั้นลง หรือใช้เวลาครบ Quota ของ Priority level นั้นๆ (ไม่รวมว่าจะเปลี่ยนไปใช้บริการอื่นและกลับมาที่บริการของธนาคารของตน) จะรอจนลด Priority ลูกค้าที่มานั้นลง และกลับมาบริการลูกค้าที่เข้ามาใหม่ หรือจะรอจนกว่า (periods) ให้ reset priority ลูกค้าทุกคนขึ้นมาที่ priority แรก แล้วให้ลูกค้า priority ต่ำ หรือรอได้ใช้บริการอีกครั้ง

2. จงอธิบายการเกิดขึ้นของ Internal Fragmentation และ External Fragmentation สำหรับการจองพื้นที่ในหน่วยความจำหลักแบบใช้พื้นที่ต่อเนื่อง (Contiguous Memory Allocation) พร้อมทั้งอธิบายถึงผลกระทบและแนวทางการจัดการว่าเหมือนหรือแตกต่างกันอย่างไร (5 คะแนน)

Internal Fragmentation เกิดขึ้นใน Contiguous Memory Allocation แบบ Fixed-Sized นั่นคือ พื้นที่ใน allocated unit ไม่ได้ถูกใช้ (พื้นที่ที่ว่าง stack กับ heap) กันนั่นเอง!

External Fragmentation เกิดขึ้นใน Contiguous Memory Allocation แบบ Flexible-sized นั่นคือ แต่ละ allocation unit ขนาดไม่เท่ากัน ทำให้ไม่สามารถเอามาใช้ได้ใน physical mem ได้อย่างมีประสิทธิภาพและเกิดข้อผิดพลาดในการใช้ memory เป็นปกติ (External Frag) ทำให้พื้นที่ว่างในหน่วยความจำจะให้เป็น Contiguous mem เป็นไปไม่ได้ (เป็นข้อบกพร่อง)

Internal Fragmentation เกี่ยวข้องกับ Segmentation คือ แบ่งทั้ง address space ตาม logical unit (e.g. code, heap, stack, etc) และเวลาแต่ละส่วนใช้ base & bound คำนวณขีดจำกัดใน physical mem ตัวแบ่งส่วนกันไว้ แต่มักเกิดข้อผิดพลาด heap, stack หรือส่วน heap, stack ออกจากกันซึ่งจะลดข้อผิดพลาด

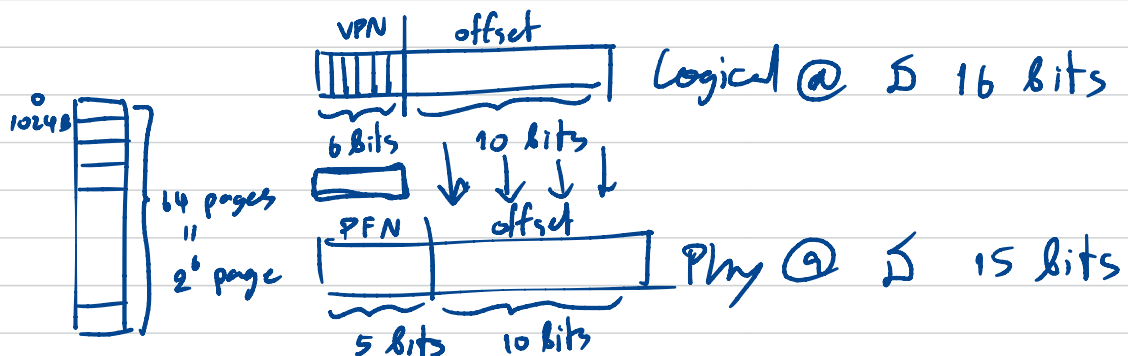
External fragmentation แก้ไข 1.1) Memory Coalescing; รวม free space Adjacent ที่อยู่ติดกันมารวมเป็น large free space
1.2) Memory Compaction; ย้าย free space มาอยู่ติดกัน (คือในแบบ Dynamic Allocation) เมื่อเกิด free space ที่ใหญ่พอให้ new mem เป็นไปได้
2) Paging ทุก page ขนาดเท่ากัน เก็บใน Physical mem หนึ่ง

3. การจองพื้นที่ในหน่วยความจำหลัก โดยใช้แนวคิดที่เรียกว่า Paging Scheme จำเป็นต้องใช้ Page Table จงอธิบายถึงความสำคัญของการมี Page Table ส่วนการเพิ่มประสิทธิภาพการทำงานของ Page Table สามารถใช้เทคนิคที่เรียกว่า Inverted Page Table จงอธิบายหลักการทำงานของ Inverted Page Table และแนวโน้มในการเพิ่มประสิทธิภาพ (5 คะแนน) (๗) (๒) (๓)

หน้าที่หลักของ page table คือการเก็บ address translation สำหรับแต่ละ virtual page ของ address space ทำให้เรารู้ว่า page อยุ่ที่ใดใน physical mem

ส่วนการของ Invert page table แทนที่จะมีหลาย page table (1 page table ต่อ 1 process) เราจะมี 1 page table ที่มี entry ของแต่ละ physical page frame โดย entry นี้จะบอกเราว่า process ใดในเครื่องใช้ page frame นี้ (ตรงกับ PID ในเครื่อง) และ virtual page ใดใน process นี้ map มาที่ phy page นี้ ในแง่ประสิทธิภาพ เป็นการลดพื้นที่ในเครื่องกับ page table แบบ table อย่างอื่น แต่ table เดียว

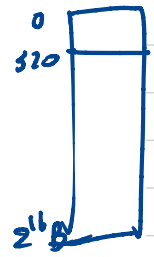
4. พิจารณาระบบปฏิบัติการที่ใช้การจัดการหน่วยความจำหลักแบบ Paging Scheme มี Logical Address 64 บิต แต่ละเพจมีขนาด 1024 ไบต์ ซึ่งทำงานคู่กับหน่วยความจำหลักที่มี Physical Address 32 เพรม จงบอกจำนวนบิตของ Logical Address และ Physical Address (2 คะแนน)



5. จงคำนวณหา Physical Address จาก Logical Address ขนาด 16 บิต เมื่อระบบปฏิบัติการใช้วิธี Segmentation เพื่อจองพื้นที่ในหน่วยความจำหลัก และมี Segment Table ดังตารางด้านล่าง โดยสมมติให้ 4 บิตบนเป็นตัวบอกเซกเมนต์ (ต้องตอบเป็นฐานสิบ) (5 คะแนน)

	Segment	Base	Size
0001	Code	32K	2K
0010	Heap	34K	2K
0011	Stack	28K	2K

32768



5.1. Logical Address เป็น 320 (เลขฐานสิบ)

5.2. Logical Address เป็น 2561 (เลขฐานสิบ)

5.3. Logical Address เป็น 12528 (เลขฐานสิบ)

$$5.1) (320)_{10} = (\underbrace{0001}_{\text{code}} \ 0100 \ 0000)_2$$

$$Phy = 32768 + 320 = 33088$$

$$5.2) (2561)_{10} = (1010 \ 0000 \ 0001)_2$$

5.3) Out of bound

6. ถ้าเปรียบเทียบระบบปฏิบัติการตัวเดียวกัน ที่มีการเลือกใช้ขนาด Page แตกต่างกันคือ ขนาดเล็กกับขนาดใหญ่ จะส่งผลกับอัตราการเกิด Page Fault แตกต่างกันอย่างไร (3 คะแนน)

7. จงแสดงตาราง Frame ของหน่วยความจำหลัก และจงคำนวณ Effective Access Time เมื่อมีสถานะการณ์การเข้าถึง Page ตามลำดับต่อไปนี้ (7 คะแนน)

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1 (นับได้ 20 ตัว)

สมมุติให้

- จำนวน Frame คือ 4
- เวลาในการเข้าถึงหน่วยความจำหลักเป็น 9 ns
- เวลาให้การเข้าถึงหน่วยความจำสำรอง 25 ms

7.1. เมื่อใช้ Page Replacement Algorithm เป็นแบบ LRU

7.2. จากข้อมูลเวลาในการเข้าถึงหน่วยความจำหลักและสำรองตามที่กำหนดไว้ ถ้าต้องการให้ EAT ซ้ำกว่าเวลาในการเข้าถึงหน่วยความจำหลักไม่เกิน 100 เท่า page fault rate ควรมีค่าเป็นอย่างไร

7.1) 7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1

7	7	7	7 ³	7 ³	3 ⁰	3 ¹	3 ²	3 ³	3 ³	3 ¹	3 ⁰	3 ¹	3 ²	3 ²	3 ³	3 ³	7 ⁰	7 ¹	7 ²
0	0	0 ²	0 ⁰	0 ¹	0 ⁰	0 ¹	0 ²	0 ³	0 ⁰	0 ¹	0 ²	0 ³	0 ³	0 ⁰	0 ¹	0 ²	0 ⁰	0 ¹	0 ²
1	1 ¹	1 ²	1 ³	1 ³	4 ⁰	4 ¹	4 ²	4 ³	4 ³	4 ³	4 ³	1 ⁰	1 ¹	1 ²	1 ⁰	1 ¹	1 ²	1 ⁰	1 ⁰
2	2 ⁰	2 ¹	2 ²	2 ²	2 ³	2 ³	2 ³	2 ³	2 ¹	2 ¹	2 ²	2 ⁰	2 ¹	2 ²	2 ¹	2 ²	2 ³	2 ³	2 ³
m	m	m	m	h	m	h	m	h	h	h	h	h	m	h	h	h	m	h	h

$$\text{miss rate} = \frac{8}{20}$$

$$\text{hit rate} = \frac{12}{20}$$

$$\text{EAT} = \text{hit rate} \left(9 \times 10^{-9} \right) + \text{miss rate} \left(25 \times 10^{-3} \right)$$

$$= 10 \text{ ms}$$

7.2 $\text{EAT} \leq 100 (9 \times 10^{-9})$

$$(1 - \text{mr}) (9 \times 10^{-9}) + \text{mr} (25 \times 10^{-3}) \leq 9 \times 10^{-7}$$

$$0.025 \text{ mr} \leq 8.91 \times 10^{-9}$$

$$\text{mr} \leq 3.564 \times 10^{-5}$$